

Continuing from [Проект цифровых чеклистов](chatgpt-conversation://6a9c6c75-76a8-83ed-8b8b-f69acf4f71ed):
Проведи глубокий ресерч идеи системы цифровых чек-листов для сети Додо Пицца. Исследуй интернет

Отличная задача. Додо Пицца известна своей мощной внутренней ERP-системой (Dodo IS), и создание для нее (или по ее образу) системы цифровых чек-листов — это амбициозный проект, требующий enterprise-подхода даже на этапе MVP.

GitHub

Ниже представлен глубокий ресерч предметной области, анализ конкурентов, архитектурные инсайты и список критических вопросов для проектирования.

1. Исследование рынка и GitHub: Аналоги и конкуренты

Коммерческие решения (Enterprise QSR Operations)

Рынок систем для управления операционкой ресторанов (QSR — Quick Service Restaurants) занят гигантами, с которыми вам предстоит концептуально конкурировать:

- **Jolt / Zenput (теперь Crunchtime) / MeazureUp:** Стандарты индустрии. Их главные фишки: иерархия франшиз, Bluetooth-термошупы, жесткий контроль геопозиции и времени, PWA/мобильные приложения для оффлайна.

CS-Cart

- **SafetyCulture (iAuditor):** Самое популярное приложение для аудитов. Гибкий конструктор форм, версионирование, мощная аналитика.

G2

- **Что у них под капотом:** Сложный движок форм (по сути, интерпретатор JSON-схем), Event Sourcing для аудита (нельзя изменить запись после отправки), мощный RBAC (доступ по ролям и локациям).

Open-Source решения (GitHub) GitHub Pages

Готового open-source решения, идеально подходящего под бизнес-процессы сетевого фастфуда с мульти-локациями и QR-кодами, не существует. Большинство репозиторий (например, для гостиниц или POS-терминалов) решают задачи продаж, а не операционного контроля.

Однако есть проекты, модули которых имеет смысл переиспользовать или изучить:

- **Form.io / ODK (Open Data Kit) / KoboToolbox:** Мощные open-source движки для создания сложных форм и сбора данных в оффлайне. Брать целиком тяжело (UI не заточен под QSR), но стоит перенять подход к хранению схем.
- **Documenso:** Open-source альтернатива DocuSign. Отличный референс того, как делать криптографически достоверный audit trail (кто, когда и с какого IP подписал/заполнил форму).

2. Архитектурные паттерны и подводные камни

Что переиспользовать, а что писать с нуля?

- **Писать с нуля:** Бизнес-логику иерархии (Сеть -> Регион -> Франчайзи -> Пиццерия), UI/UX самого чек-листа (он должен быть максимально "тупым" и быстрым для линейного персонала), систему роутинга QR-кодов.
- **Переиспользовать:** UI-библиотеки (shadcn/ui или Tailwind), ORM (Prisma/Drizzle), сервисы аутентификации (Keycloak или NextAuth/Auth.js — для AI агента это проще), и движки PWA/Service Workers (например, Workbox).

Ключевые паттерны и подводные камни (Pitfalls)

1. QR-код -> Веб-чеклист (Deep Linking & Context):

1. Паттерн: QR-код должен содержать не просто ссылку на форму, а контекстный payload, например: `[https://app.com/c?loc=123&zone=kitchen&asset=fridge_1](https://app.com/c?loc=123&zone=kitchen&asset=fridge_1)`.
2. Подводный камень: Пользователь сканирует QR, но не авторизован или в оффлайне. PWA должен уметь кешировать интент сканирования, просить пин-код/логин и затем открывать нужную форму из IndexedDB.

2. Versioned Checklist Templates (Версионирование):

1. Паттерн: Использование JSONB в PostgreSQL. Шаблон чек-листа — это JSON-схема. Заполненный чек-лист сохраняет внутри себя снимок (snapshot) структуры шаблона на момент заполнения.
2. Подводный камень: Если реляционно связать ответы с вопросами, то при удалении или изменении вопроса в шаблоне, поедет вся историческая аналитика. **Никогда не мутируйте шаблоны** — только создавайте новую версию (v1 -> v2).

3. Offline / PWA (Синхронизация):

1. Паттерн: Local-first архитектура. Данные пишутся в IndexedDB (через RxDB, WatermelonDB или просто Dexie.js), а Background Sync API отправляет их на сервер при появлении сети.
2. Подводный камень: Конфликты времени (Clock Skew). У сотрудников на телефонах может быть сбито время. Сервер должен фиксировать время получения (`server_created_at`) помимо времени устройства (`device_created_at`).

4. Multi-location Hierarchy & Permissions:

1. Паттерн: ABAC (Attribute-Based Access Control) в комбинации с RBAC. Роль: "Менеджер". Атрибут: "Пиццерия №45".
2. Подводный камень: Каскадное наследование. Управляющая компания создает шаблон "Закрытие смены". Может ли франчайзи добавить туда 2 своих локальных вопроса? Если да, как потом сводить глобальную аналитику?

5. Role/User Audit Trail (Защита от фрода):

1. Паттерн: Event Sourcing или Append-only таблицы для логов.
2. Подводный камень: "Галерейный фрод" (загрузка старого фото чистого пола вместо реального снимка). Нужно запрещать `<input type="file">` и использовать WebRTC/Camera API для захвата фото только в реальном времени с наложением вотермарки (Timestamp + GPS).

6. Multilingual UI/Content:

1. Паттерн: UI-строки переводятся через i18n (например, `next-intl`), а контент шаблонов (названия вопросов) хранится в формате словаря в JSON: `{"en": "Clean floor", "ru": "Помой пол", "uz": "Po'lni yuving"}`.

3. Требования к стеку, деплою и AI-разработке

Поскольку разработка будет вестись AI-агентом (Cursor/Devin) для пилота на личном сервере с последующей миграцией (вероятно, в AWS/Yandex Cloud или инфраструктуру Dodo), есть жесткие критерии:

- **Стек для AI-агента:** Агенты лучше всего пишут код на самых популярных, хорошо задокументированных фреймворках с сильной типизацией.
 - Рекомендация: **Next.js (App Router)** (Fullstack) или **React (Vite) + NestJS/Go** (если бэкэнд сложный). Язык: строго **TypeScript**.
 - БД: **PostgreSQL** (идеально для JSONB и иерархий Ltree) + Prisma или Drizzle ORM (AI их обожает).
- **Deployment (Self-hosted -> Cloud):**
 - Пилот должен разворачиваться через **Docker Compose** (Postgres + App + Redis + MinIO для S3-совместимого хранения фото). Это позволит перенести систему в облачный Kubernetes без изменения кода (MinIO заменится на AWS S3, локальный Postgres на managed-базу).
- **Security & Auditability:**
 - Данные не должны удаляться (`Soft Delete`).
 - Токены доступа (JWT) должны иметь короткий срок жизни + Refresh токены.
 - Ограничение по IP или геолокации для предотвращения заполнения чеклистов из дома.

4. КРИТИЧЕСКИ ВАЖНЫЕ ВОПРОСЫ (Clarifying Questions)

Перед тем как я начну проектировать архитектуру и писать промпты для AI, ответь на следующие вопросы:

Блок 1: Бизнес-логика и Иерархия (Domain Rules)

1. **Кастомизация на местах:** Будет ли у менеджера конкретной пиццерии (или франчайзи) право редактировать глобальные шаблоны от Управляющей Компании (УК) или добавлять в них свои локальные пункты?

2. **Сценарии выполнения:** Чек-лист заполняется одним человеком за один проход, или это коллаборативный процесс (например, утреннюю смену параллельно открывают два кассира на разных телефонах, заполняя один общий чек-лист)?
CS-Cart
3. **Связь с сотрудниками:** Будет ли интеграция с текущей базой сотрудников (например, через API Dodo IS / Active Directory), или мы делаем изолированную систему с собственной базой пользователей и логинами/пин-кодами?

Блок 2: Оффлайн и UI (User Experience)

4. **Уровень оффлайна:** Насколько глубокий оффлайн нужен? Достаточно ли просто сохранить результаты заполнения при обрыве сети в холодильнике (до 1 часа), или приложение должно уметь загружать новые шаблоны и работать автономно целыми днями?
5. **Антифрод (Защита от обмана):** Нужна ли обязательная фиксация геолокации (GPS) и защита от загрузки фотографий из галереи телефона (требование делать фото только через камеру в приложении)?
6. **Многоязычность данных:** На разных языках должен быть только интерфейс кнопок, или сами названия задач в чеклистах тоже должны переводиться динамически в зависимости от профиля сотрудника? (Например, менеджер пишет на русском, а повар видит задачу на узбекском).

Блок 3: Интеграции и Реакции (Event Driven)

7. **Инциденты (Action Items):** Что должно происходить, если пункт чеклиста помечен как "Не выполнено/Сломано"? Должна ли система автоматически создавать инцидент/задачу на починку, или достаточно просто зафиксировать провал в истории?
8. **QR-архитектура:** QR-коды статичны (распечатаны и наклеены на оборудование на годы) или генерируются динамически (на экранах)?

GitHub Pages

Блок 4: Инфраструктура и Миграция

9. **Хранение медиа:** Чек-листы подразумевают фото-отчеты. Даже на этапе пилота база быстро распухнет от картинок. Согласен ли ты сразу использовать S3-совместимое хранилище (например, MinIO в Docker), чтобы не хранить файлы на диске сервера?

Жду твоих ответов, чтобы сформировать точную архитектурную спецификацию, на основе которой твой AI-агент безошибочно напишет код.

апдейт: чек лист именно что просто интерактивный чек лист. все. Фото не планировалось, но как идея про некий финал смены - вполне.

Упрощение функционала до чистого интерактивного чек-листа существенно облегчает архитектуру, ускоряет разработку AI-агентом и избавляет от необходимости

поднимать S3-хранилища для медиафайлов. Фокус полностью смещается на мгновенный UI, оффлайн-устойчивость и надежную историю выполнения.

Что меняется в архитектуре

- **Хранение:** Не нужны S3/MinIO и сложные пайплайны обработки фото. Вся система полностью уместается в одной базе PostgreSQL. Опциональное фото финала смены при необходимости сохраняется прямо в БД или внешнюю S3-ссылку простым полем.
- **Производительность:** Чек-листы весят килобайты. Интерфейс через PWA загружается мгновенно даже при слабом Wi-Fi на кухне или в холодильной камере.
- **Антифрод:** Сложная защита от загрузки старых фото из галереи убирается. Контроль честности сводится к фиксации связки `User ID + Server Timestamp + Device Timestamp + QR Location ID`.

Критический функционал MVP

- **QR-роутинг:** Сканирование QR-кода на оборудовании или в зоне (например, «Упаковка» или «Касса») сразу открывает соответствующий чек-лист.
- **Иерархия и доступ:** Назначение чек-листов по пиццериям, зонам и ролям (менеджер, пиццамейкер, кассир).
- **Версионирование шаблонов:** Изменение пунктов чек-листа сегодня не ломает исторические отчеты за прошлые месяцы (структурные снимки шаблонов в JSONB).
- **Audit Trail:** Фиксация логов действия (кто, когда и в каком порядке отметил выполнение пунктов).

Оптимальный стек для AI-разработки (минимальная сложность)

- **Fullstack:** Next.js (TypeScript, App Router) — единый репозиторий для UI и бэкенда, идеален для генерации кода агентами (Cursor/Devin).
- **База данных:** PostgreSQL + Drizzle ORM — минимальный оверхед, отличная поддержка типов и JSONB.
- **Авторизация:** JWT-токены с быстрой авторизацией по ПИН-коду сотрудника или смены.
- **PWA / Offline:** Dexie.js (IndexedDB) для локального сохранения состояния галочек при потере связи + Service Worker для кеширования UI.

Поскольку оффлайн актуален для закрытых зон пиццерии, отправлять ли состояние галочек на сервер мгновенно при каждом клике, или чек-лист должен отправляться единым пакетом только по кнопке «Завершить смену»?

1. думаю что должна быть возможность создать чеклист под конкретную страну. т.е. должен быть арендатор для страны, внутри которого как раз будут чеклисты и пиццерии. но это надо будет обсуждать

Принято, эти вводные существенно упрощают UX и формируют четкую архитектуру, идеальную для быстрой разработки.

Иерархия и Мультитенантность (Multi-tenancy)

- **Структура данных:** Country (Tenant) -> Partner/Franchisee -> Store (Location) -> Station (Зона).
- **Шаблоны чеклистов:** Страна задает глобальные стандарты. При создании чеклиста задается тип формата пиццерии (Standard/Full с разделением на Утро/Вечер или Compact/Express для смен из 2 человек).
- **Локализация:** База данных хранит текстовые поля задач в виде словарей: `title: { "ru": "Очистить печь", "kk": "Пешті тазалау", "en": "Clean the oven" }`. Язык выбирается автоматически по арендатору страны или переключателем в UI.

Быстрый UX и Вход (Zero-Friction Auth)

- **Авторизация по PIN:** Сотрудникам не нужен логин в Dodo IS.
- **Сценарий работы:**
 1. Сотрудник сканирует QR-код станции (наклейка на стене или экран планшета).
 2. Приложение определяет пиццерию и зону, показывают список сотрудников смены на этой локации.
 3. Сотрудник выбирает свое имя и вводит 4-значный PIN-код.
 4. Чек-лист открывается мгновенно. Сессия автоматически сбрасывается через 15 минут неактивности.

Сигнализация и Инциденты (Alerting & Action Items)

- **Мягкая фиксация:** Если пункт отмечается как «Не выполнено / Сломано», система не блокирует смену, а запрашивает короткий текстовый комментарий.
- **Алерты:** Пункты с флагом `IsCritical` при провале создают запись в таблице `Incidents` и отправляют алерт (через Webhook/Telegram-бота или внутренний дашборд) управляющему пиццерии.
- **Финал смены:** При закрытии смены формируется итоговый отчет (Summary), куда попадают все проблемные пункты за день.

QR-коды и Фото-финалы

- **Статичные QR:** Содержат постоянную ссылку (`/scan?store=101&station=kitchen`). Печатаются один раз и наклеиваются на оборудование.
- **Динамические QR:** Выводятся на экран планшета станции с зашитым временным токеном.
- **Фото в финале:** 1–2 фото только при закрытии смены (чистота, выключенное оборудование). Изображения автоматически сжимаются на клиенте до WebP (до

150 КБ) и передаются на бэкенд без необходимости сразу поднимать S3-хранилище.

Делаем ли мы полноценный визуальный конструктор шаблонов (drag-and-drop) в админ-панели для управляющей компании сразу в MVP, или на первом этапе шаблоны будут загружаться через простой JSON-конфиг / базовую форму?

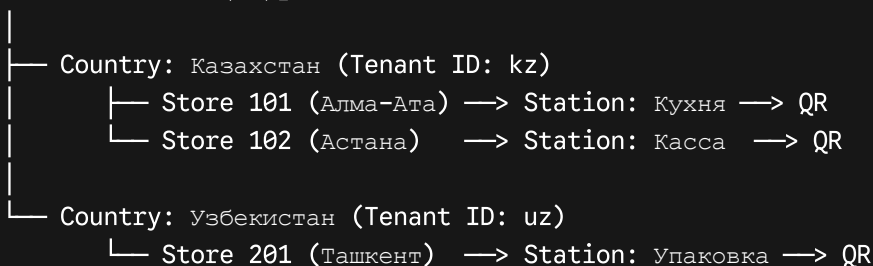
1. УК задает стандарт. партнеры на местах могут адаптировать. это мы еще обсудим с командой по приоритетам и задачам.
2. язык можем как-то брать от языка устройства? 

Архитектурная концепция системы цифровых чек-листов адаптирована под требования максимального упрощения UX (Zero-Friction), гибкой мультитенантности и высокой скорости разработки через AI-агента.

1. Доменная модель и Иерархия (Multi-Tenant)

Иерархия исключает сущность «Партнер» из основного дерева доступа, сохраняя прямую вертикаль Управляющая Компания (УК) → Страна → Пиццерия.

[Управляющая компания (УК)]



- **Управляющая Компания (УК):** Создает глобальные стандарты и базовые шаблоны чек-листов.
- **Страна (Тенант):** Изолирует настройки, дефолтный язык и локальные адаптированные шаблоны.
- **Пиццерия (Store):** Конкретная точка, к которой привязывается факт выполнения смены.
- **Станция (Station):** Зона внутри пиццерии (Печь, Касса, Кухня, Открытие/Заккрытие), имеющая свой QR-код.

2. Zero-Friction Flow и Детекция нарушений (Анти-проёб)

Снятие требования по авторизации сотрудников полностью убирает трение при заполнении: сотрудник не тратит время на логин, ввод пин-кодов или поиск профиля.

Сценарий работы

1. Сотрудник сканирует QR-код станции (например, `/scan?store_id=101&station=kitchen`).
2. Приложение мгновенно открывает актуальный чек-лист для этой станции и текущей смены (Утро/Вечер).
3. При нажатии на галочку система логирует событие: `Store ID + Station ID + Question ID + Client Timestamp + Server Timestamp`.

Автоматическое выявление нарушений (Аналитика)

Так как конкретный пользователь не авторизован, контроль дисциплины переносится на временные паттерны:

- **Импульсное / Пакетное проставление (Batch completion):** Если 15 пунктов чек-листа отмечаются за 5 секунд, смене присваивается статус `Flagged: Batch Checked` (формально заполнено «для галочки»).
- **Заполнение не вовремя (Late completion):** Утренний чек-лист открыт или закрыт в 16:00. Система ставит метку `Flagged: Out of Window`.
- **Пропущенные смены (Gaps):** Если за дату по станции нет ни одной записи, на дашборде управляющего пиццерия подсвечивается красным.

3. Автоматическая локализация (Multi-language)

Система определяет язык на основе заголовка `Accept-Language` или `navigator.language` устройства пользователя.

Формат хранения текста в PostgreSQL (JSONB)

Каждый пункт чек-листа хранит названия и подсказки в формате словаря:

JSON



```
{
  "title": {
    "ru": "Проверить температуру в холодильнике",
    "kk": "Тоңазытқыштың температурасын тексеріңіз",
    "uz": "Sovutgich haroratini tekshiring",
    "en": "Check refrigerator temperature"
  },
  "description": {
    "ru": "Норма: от +2°C до +4°C",
    "kk": "Норма: +2°C-тан +4°C-қа дейін"
```


Логика рендеринга:

1. UI запрашивает язык устройства (например, `kk-KZ` → `kk`).
2. Если перевода на `kk` нет в JSONB, применяется язык Страны-Тенанта.
3. Если и его нет — используется дефолтный системный язык (`ru`).

4. Веб-интерфейс Админ-панели (Конструктор Шаблонов)

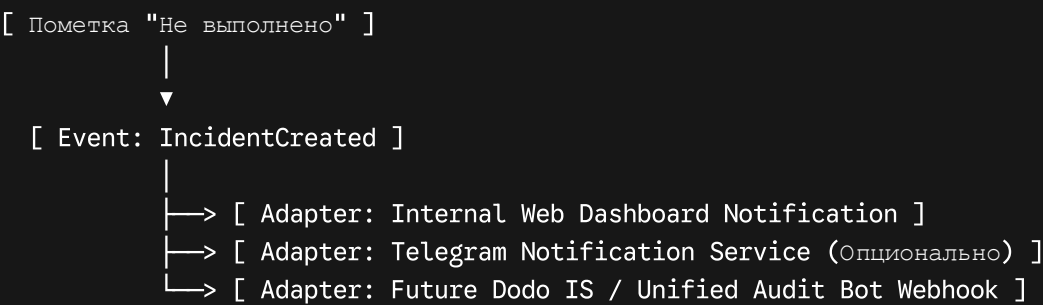
Вместо сложного Drag-and-Drop на этапе MVP используется таблично-формовый конструктор. Он прост в реализации AI-агентом и удобен для наполнения контентом.

Структура экрана создания чек-листа

+-----+			
Новый шаблон чек-листа			
+-----+			
Название (Внутреннее):			
Страна: [Казахстан v]			
Станция: [Кухня v]			
Период: [Утренний (08:00 - 10:00) v]			
+-----+			
Пункты чек-листа:			
1. [≡] Название (RU): [Включить печи и вытяжку]			
Название (KK): [Пештерді және сорғышты қосыңыз]			
Тип ответа: (o) Да/Нет () Число () Текст			
[X] Критичный пункт (Требуется комментарий при отмене)			
[Добавить описание/подсказку]			
2. [≡] Название (RU): [Замерить температуру фритюра]			
Тип ответа: () Да/Нет (o) Число [Мин: 160] [Макс: 180]			
[] Критичный пункт			
[+ Добавить пункт]			
+-----+			
[Сохранить как черновик]			
[Опубликовать версию v1.2]			
+-----+			

5. Модульная система уведомлений

Чтобы не создавать перегрузку телеграм-ботами, архитектура использует шину событий (Event Bus / Webhooks). Система фиксирует события и передает их в подключенные адаптеры.



Если реакция на провал не нужна мгновенно, данные просто накапливаются и выводятся на сводном дашборде за день.

6. Стек и инструкции для разработки AI-агентом (Cursor / Devin)

Структура проекта оптимизирована для генерации кода языковыми моделями без потери контекста.

Компонент	Технология	Причина выбора для AI
Framework	Next.js (App Router, Server Actions)	Единый TypeScript стек, минимальный шаблонный код.
UI Kit	Tailwind CSS + shadcn/ui	AI идеально генерирует компоненты shadcn из коробки.
ORM / DB	PostgreSQL + Drizzle ORM	Drizzle даёт полную типизацию и легкую работу с JSONB.
State / Offline	Dexie.js (IndexedDB wrapper)	Простая работа с локальным кешем без бойлерплейта.
...		

Стартовая схема БД для генерации (Drizzle Schema)

TypeScript

⬇️📄

```
// schema.ts
import { pgTable, uuid, text, timestamp, jsonb, boolean, integer } from 'drizzle-orm/pg-co

export const stores = pgTable('stores', {
```

```

id: uuid('id').primaryKey().defaultRandom(),
countryCode: text('country_code').notNull(), // 'ru', 'kz', 'uz'
name: text('name').notNull(),
});

export const checklistTemplates = pgTable('checklist_templates', {
  id: uuid('id').primaryKey().defaultRandom(),
  version: integer('version').notNull().default(1),
  title: text('title').notNull(),
  station: text('station').notNull(), // 'kitchen', 'cashier'
  storeFormat: text('store_format').notNull(), // 'full', 'express'
  items: jsonb('items').notNull(), // [{ id, title: {ru, kk}, isCritical, type }]
  createdAt: timestamp('created_at').defaultNow(),
});

export const submissions = pgTable('submissions', {
  id: uuid('id').primaryKey().defaultRandom(),
  storeId: uuid('store_id').references(() => stores.id),
  templateId: uuid('template_id').references(() => checklistTemplates.id),
  station: text('station').notNull(),
  responses: jsonb('responses').notNull(), // [{ itemId, status: 'ok'|'fail', comment, cli
  isBatchChecked: boolean('is_batch_checked').default(false),
  startedAt: timestamp('started_at').notNull(),
  completedAt: timestamp('completed_at').defaultNow(),
});

```

Порядок Промптов для AI-агента (Roadmap реализации)

1. **Prompt 1 (Database & API):** Разверни Next.js проект с Drizzle ORM и PostgreSQL. Создай схемы БД для `stores`, `checklist_templates` и `submissions`. Напиши API для CRUD операций шаблонов.
2. **Prompt 2 (Admin UI):** Создай интерфейс админки на shadcn/ui для управления шаблонами чек-листов с поддержкой мульти-язычных полей (JSONB) и флагов критичности.
3. **Prompt 3 (Public Checkup Flow):** Напиши публичную PWA-страницу `/scan?storeId=X&station=Y`. Страница должна автоматически определять язык устройства, загружать нужный шаблон и давать возможность проставлять галочки без авторизации.
4. **Prompt 4 (Analytics & Anomaly Detection):** Добавь логику расчета времени между кликами. Если разница между заполнением всей формы < 10 секунд, ставь флаг `isBatchChecked = true`. Создай отчетную страницу для управляющего с фильтром по датам и точкам.

1. по поводу настроек с проебами, не вовремя открыто и прочее тут неплохо бы просто подсвечивать в вебе, чтобы была возможность обратить внимание. никаких санкций не планируется.

1. Мягкая подсветка и Геолокация (Антифрод без блокировок)

Вся аналитика аномалий работает на уровне визуальных меток (Badges) в веб-интерфейсе управляющего. Никаких авто-штрафов или запретов отправки — только подсветка для внимания.

Геолокация

При сканировании QR-кода браузер запрашивает координаты устройства (`navigator.geolocation`).

- В карточке пиццерии задаются координаты (*Lat, Lon*) и радиус геозоны (например, 150 метров, чтобы учесть плохой GPS внутри зданий).
- Если координаты устройства выходят за пределы геозоны или пользователь запретил доступ к GPS, смене присваивается мягкий флаг `Flagged: Geo Mismatch` или `Flagged: No GPS`.
- Заполнение не блокируется: сотрудник делает свою работу, но управляющий на дашборде видит значок гео-расхождения.

Визуальные статусы в истории смен

- Зеленый статус:** Заполнено вовремя, в пределах пиццерии, с нормальным интервалом кликов.
- Желтый тег (Time):** `Заполнено с задержкой` (например, утренний чек-лист закрыт в 14:00).
- Желтый тег (Geo):** `Вне локации` (расстояние от пиццерии > 150 м).
- Оранжевый тег (Speed):** `Быстрое заполнение` (все пункты отмечены менее чем за 10 секунд).

2. Гибкая и масштабируемая локализация (i18n)

Чтобы не править код при добавлении новых языков (например, татарского или монгольского), локализация разделяется на два независимых слоя:



1. **Системный UI (Кнопки, статусы, системные ошибки):** Хранится в JSON-файлах переводов. Добавление нового языка UI — это просто добавление файла `mn.json` (монгольский) без изменения логики компонентов.
2. **Контент чек-листов (Названия вопросов и подсказки):** Хранится динамически в базе данных в формате `JSONB`. В админ-панели создатель чек-листа сам добавляет переводы для каждого пункта. Если перевода на язык устройства нет, система автоматически переключается на язык страны (Tenant Default Language) или на русский.

3. Модульная шина уведомлений (Notification Connectors)

Система проектируется по паттерну **Observer / Event Bus**. Сам модуль выполнения чек-листов ничего не знает о том, куда отправляются сообщения — он просто генерирует событие в систему.

События системы (System Events)

- `IncidentCreated` — пункт помечен как «Не выполнено/Сломано».
- `ShiftCompletedWithErrors` — смена закрыта с незакрытыми проблемами.
- `ChecklistLateAlert` — чек-лист не открыт в назначенный временной интервал.

Архитектура коннекторов (Pluggable Handlers)

Каждый канал связи — это отдельный легкий модуль-адаптер:

TypeScript



```
// Контракт для любого будущего коннектора
interface NotificationConnector {
  name: string;
  send(event: SystemEvent): Promise<void>;
}

// Пример адаптера для Webhook (универсальный)
class WebhookConnector implements NotificationConnector {
```

```

name = "Generic Webhook";
async send(event: SystemEvent) {
  await fetch(process.env.WEBHOOK_URL, { method: 'POST', body: JSON.stringify(event) });
}
}

// Пример адаптера для Telegram (Подключается позже одной строчкой)
class TelegramConnector implements NotificationConnector {
  name = "Telegram Bot";
  async send(event: SystemEvent) { /* Отправка в бот */ }
}

```

Чтобы подключить сирену в пиццерии, Telegram-бота или единую систему аудита Dodo, достаточно написать новый адаптер и зарегистрировать его в конфиге.

4. Веб-интерфейс конструктора (Конструкция UI)

Конструктор админ-панели сделан без сложного DND (Drag-and-Drop), а по принципу линейного редактора блоков. Ниже представлен промпт для генерации этого компонента в Claude / Cursor и HTML-структура.

Готовый промпт для AI-агента (Cursor / Claude)

Prompt for AI:

"Create a React component (`ChecklistEditor.tsx`) using Tailwind CSS and Lucide React icons for a Checklist Builder Admin UI.

Requirements:

1. Header with Metadata: Checklist Title (input), Country Selector (dropdown), Store Format Selector ('Standard' | 'Compact'), Station Selector ('Kitchen' | 'Cashier' | 'Delivery').
2. List of Items/Questions. Each item card should contain:
 - Reorder controls (Up / Down arrows).
 - Language tab switcher for question text (RU | KK | UZ | EN).
 - Question Title input field for the selected language.
 - Response Type selector: 'Yes/No Checkbox', 'Numeric Range', 'Text Comment'.
 - Toggle switch: 'Is Critical' (highlights item if marked as failed).
 - Button to remove the item.
3. Bottom actions: '+ Add New Item' button, 'Save Draft' button, 'Publish Version' button.

Make the UI clean, modern, with subtle borders, following shadcn/ui design language."

Графический HTML/Tailwind прототип конструктора

(Этот фрагмент кода можно скопировать и запустить или передать AI как референс)

```
<div class="max-w-4xl mx-auto p-6 bg-slate-50 min-h-screen font-sans">
  <!-- Шапка шаблона -->
  <div class="bg-white p-6 rounded-xl border border-slate-200 shadow-sm mb-6 space-y-4">
    <h2 class="text-xl font-bold text-slate-800">Редактор шаблона чек-листа</h2>
    <div class="grid grid-cols-1 md:grid-cols-3 gap-4">
      <div>
        <label class="block text-xs font-semibold text-slate-500 uppercase mb-1">Название<
        <input type="text" value="Утреннее открытие кухни" class="w-full border rounded-lg
      </div>
      <div>
        <label class="block text-xs font-semibold text-slate-500 uppercase mb-1">Страна (Т
        <select class="w-full border rounded-lg p-2 text-sm bg-white">
          <option>Казахстан (KZ)</option>
          <option>Узбекистан (UZ)</option>
        </select>
      </div>
      <div>
        <label class="block text-xs font-semibold text-slate-500 uppercase mb-1">Станция</
        <select class="w-full border rounded-lg p-2 text-sm bg-white">
          <option>Кухня</option>
          <option>Упаковка</option>
          <option>Касса</option>
        </select>
      </div>
    </div>
  </div>

  <!-- Список вопросов -->
  <div class="space-y-4 mb-6">
    <!-- Элемент вопроса 1 -->
    <div class="bg-white p-5 rounded-xl border border-slate-200 shadow-sm relative group">
      <div class="flex items-center justify-between mb-3 border-b pb-2">
        <div class="flex items-center space-x-2">
          <span class="bg-slate-100 text-slate-600 text-xs font-bold px-2 py-1 rounded">#1
          <!-- Табы языков ввода -->
          <div class="flex space-x-1 bg-slate-100 p-1 rounded-lg text-xs font-medium">
            <button class="px-2 py-0.5 bg-white rounded shadow-sm text-orange-600 font-bol
            <button class="px-2 py-0.5 text-slate-500">KK</button>
            <button class="px-2 py-0.5 text-slate-500">UZ</button>
          </div>
        </div>
        <button class="text-red-500 hover:text-red-700 text-xs font-semibold">Удалить</but
      </div>

      <div class="space-y-3">
        <div>
          <label class="block text-xs text-slate-400 mb-1">Текст пункта (RU)</label>
          <input type="text" value="Проверить маркировку на соусах в холодильнике" class="
        </div>

        <div class="flex flex-wrap items-center justify-between gap-4 pt-2">
```



```

<div class="flex items-center space-x-4">
  <label class="text-xs font-medium text-slate-600">Тип ответа:</label>
  <label class="inline-flex items-center text-xs"><input type="radio" name="t1"
  <label class="inline-flex items-center text-xs"><input type="radio" name="t1"
</div>

<label class="inline-flex items-center cursor-pointer">
  <input type="checkbox" checked class="sr-only peer">
  <div class="w-9 h-5 bg-slate-200 peer-focus:outline-none rounded-full peer pee
  <span class="ml-2 text-xs font-semibold text-slate-700">Критичный пункт</span>
</label>
</div>
</div>
</div>
</div>

<!-- Кнопки действий -->
<div class="flex items-center justify-between">
  <button class="bg-slate-200 hover:bg-slate-300 text-slate-700 px-4 py-2 rounded-lg tex
    + Добавить пункт
  </button>
  <div class="space-x-2">
    <button class="px-4 py-2 text-sm text-slate-600 font-semibold">Сохранить черновик</b
    <button class="bg-orange-500 hover:bg-orange-600 text-white px-5 py-2 rounded-lg tex
      Опубликовать
    </button>
  </div>
</div>
</div>
</div>

```

давай про веб интерфейс сразу поговорим. Я вижу так: у нас будут вкладки: статистика , менеджер чек листов, настройки
отдельно еще вкладка админа чтобы выдавать

Предложенная структура из 4 вкладок идеально ложится на доменную модель и закрывает все потребности: от суперадминистрирования до ежедневного контроля операционки.

1. Статистика (Операционный дашборд)

Главный экран для управляющих пиццерий, франчайзи и аудиторов УК.

- **Фильтры:** Страна → Пиццерия → Станция → Диапазон дат → Статус (Завершено, Пропущено, С нарушениями).

- **Сводка (KPIs):** % затянутых/пропущенных смен, количество зафиксированных инцидентов (невыполненных критических пунктов).
- **Лента смен (Shift Feed):** Список всех заполнений с цветовой индикацией нарушений:
 - **Зеленый** — заполнено идеально.
 - **Желтый (Time)** — утренний чек-лист открыт или закрыт с опозданием.
 - **Желтый (Geo)** — заполнено за пределами геозоны пиццерии.
 - **Оранжевый (Speed)** — проставлено слишком быстро (< 10 секунд на все галочки).
- **Детализация:** Клик по смене открывает полный протокол зафиксированных ответов с временем клика по каждому пункту и комментариями сотрудника.

2. Менеджер чек-листов (Конструктор шаблонов)

Рабочее место методистов УК и менеджеров стран, где создаются и редактируются стандарты.

- **Список шаблонов:** Сгруппирован по станциям и форматам (Стандарт / Экспресс).
- **Редактор пунктов:**
 - Тексты вопросов на нескольких языках (RU, KK, UZ и др.).
 - Переключатель типа ответа (Да/Нет, Числовое значение, Комментарий).
 - Флаг «Критичный пункт» (требуется обязательный комментарий при отмене и отправляет алерт).
- **Версионирование:** Кнопка «Опубликовать новую версию». Старые заполненные смены остаются привязаны к той версии шаблона, которая действовала на момент заполнения.

3. Настройки (Справочники и Интеграции)

Конфигуратор рабочего окружения для конкретной страны или точки.

- **Управление пиццериями:** Добавление точек, их адресов, GPS-координат и радиуса допустимого заполнения (например, 150 метров).
- **Управление станциями:** Генератор QR-кодов для распечатки или вывода на планшеты (**Кухня** , **Упаковка** , **Касса**).
- **Коннекторы уведомлений:** Поле для ввода Webhook URL (или токена Telegram-бота) для отправки алертов о проваленных критических пунктах.

4. Суперадмин (Управление доступом и Странами)

Верхнеуровневый модуль, доступный только глобальным администраторам системы.

- **Управление арендаторами (Странами):** Создание новых стран (Казахстан, Узбекистан, Сербия и т.д.) с назначением дефолтного языка страны.

- **Управление пользователями панели:** Создание аккаунтов для сотрудников УК и менеджеров на местах.
- **Назначение ролей (RBAC):**
 - **Superadmin** — видит всё, управляет странами и глобальными настройками.
 - **Country Manager** — видит только свою страну, редактирует локальные шаблоны и смотрит аналитику по своим точкам.
 - **Store Manager** — видит только свою пиццерию и статистику по ней (без доступа к редактированию шаблонов).

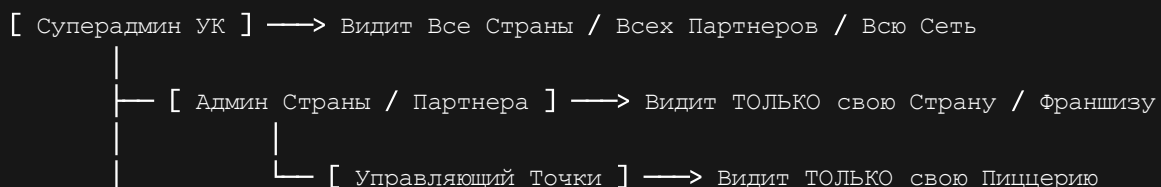
Архитектурная деталь: Динамический интерфейс по ролям

При таком разделении не нужно делать разные админки. Интерфейс остается единым, но вкладка «Суперадмин» просто скрывается для обычных пользователей, а в остальных вкладках данные автоматически фильтруются по привязанной к пользователю стране и пиццерии.

я думаю что суперадмин нужен только менеджеру УК и менеджеру от партнера. и при этом действительно у нас у СУПЕРАДИНА возможность видеть tenants всех подключившихся стран, а у менеджера от партнера возможность видеть только себя

Отличная корректировка. Это классическая модель **Multi-Tenant RBAC** с разграничением области видимости (Scope). При такой схеме интерфейс остаётся единым для всех, но содержимое страниц и доступные действия автоматически фильтруются в зависимости от роли и привязки пользователя.

Обновленная модель ролей и прав доступа



Роль	Вкладка «Администрирование»	Вкладка «Менеджер чек-листов»	Вкладка «Статистика»
Суперадмин УК	Видит все страны, создает тенанты, выдает доступы партнерам и менеджерам стран.	Создает и публикует глобальные стандарты чек-листов для всех стран.	Полная аналитика всей сети (всех стран и точек).
Админ Страны / Партнера	Управляет пользователями только своей страны (выдает доступы управляющим пиццерий).	Адаптирует глобальные шаблоны под свою страну / создает локальные чек-листы.	Аналитика по точкам своей страны/фрагментам .
Управляющий Точки	Скрыта	Только просмотр назначенных на точку шаблонов.	Аналитика только по своей пиццерии .
...			

Как это реализуется в коде и архитектуре

1. Единая таблица пользователей (Users) с понятным скоупом:
У каждого пользователя в базе фиксируется роль и область доступа:

TypeScript



```
type User = {
  id: string;
  email: string;
  role: 'SUPERADMIN_HQ' | 'PARTNER_ADMIN' | 'STORE_MANAGER';
  countryId?: string; // null для SUPERADMIN_HQ, заполнено для PARTNER_ADMIN
  storeId?: string;    // заполнено для STORE_MANAGER
};
```

2. Автоматическая фильтрация запросов (Row-Level Security):
Бэкенд при каждом запросе к БД автоматически подставляет фильтр:
 1. Если зашел Суперадмин УК → `SELECT * FROM submissions;` (показать всё).

2. Если зашел Админ Казахстана → `SELECT * FROM submissions WHERE country_id = 'kz';`.

3. Если зашел Управляющий Точки 101 → `SELECT * FROM submissions WHERE store_id = '101';`.

3. Унифицированный UI:

Разработчику не придется верстать разные кабинеты. Вкладка «Администрирование» у Суперадмина показывает список стран, а у Админа Партнера — список менеджеров его пиццерий.

Финальный чек-лист готовности к проектированию

Все ключевые решения по продукту приняты:

- **Архитектура:** Next.js + PostgreSQL + Drizzle ORM + Tailwind/shadcn (идеально для AI).
- **Авторизация:** PIN-коды / Zero-friction для линейного персонала, полноценный логин с ролями (RBAC) для админки.
- **Иерархия:** УК → Страна/Тенант → Пиццерия → Станция.
- **Антифрод:** Мягкая фиксация аномалий (скорость кликов, GPS, задержки) без блокировок.
- **Интерфейс:** 4 вкладки (Статистика, Менеджер чек-листов, Настройки, Управление доступом).
- **Локализация:** Системный UI через i18n, контент чек-листов через JSONB в БД.
- **Уведомления:** Модульная шина событий (Event Bus) с коннекторами.

Если концепция полностью утверждена, следующим шагом можно сформировать **техническое задание и пошаговый план промптов (Roadmap)** для твоего AI-агента (Cursor / Devin), чтобы начать пошаговую генерацию кода.

я думаю что управляющему точки не нужен вообще админ доступ. только дашборд со своей статистикой, и только настройки чек листов.

Абсолютно логично. Чем меньше кнопок и вкладок у управляющего пиццерией, тем проще ему работать и тем меньше риск случайных ошибок. Ему нужен только рабочий инструмент для контроля смен, а не пульт управления системой.

Уточненный интерфейс для «Управляющего точки»

Когда управляющий логинится в систему, он видит максимально аскетичный интерфейс из двух вкладок:

1. Дашборд (Статистика):

1. Лента заполненных чек-листов только по его пиццерии.
2. Подсветка аномалий (заполнено вне пиццерии, заполнено за 5 секунд, пропущена смена).
3. Процент успешных смен за неделю/месяц.

2. Настройки станций и QR-кодов:

1. Список станций его пиццерии (Кухня, Касса, Холодильник).
2. Возможность выбрать, какой именно шаблон чек-листа (из спущенных сверху от УК/Партнера) привязан к конкретной станции.
3. Кнопка «Распечатать QR-код» или «Вывести QR на экран» для каждой станции.

Вкладки «Менеджер чек-листов» (редактор) и «Управление доступом» от него полностью скрыты.

Итог: Архитектура полностью согласована

У нас сложилась кристально чистая, современная и легко масштабируемая концепция. Мы обошли главные ловушки (сложная авторизация для поваров, жесткие блокировки, путаница с версиями шаблонов) и сделали систему, которая будет помогать бизнесу, а не тормозить его.

Готов ли ты переходить к разработке?

Если да, я прямо сейчас сгенерирую для тебя **Master Roadmap & Prompts (Мастер-план промптов)**. Это будет структурированный документ, который ты сможешь по частям скармливать своему AI-агенту (Cursor, Claude или Devin), чтобы он начал писать код шаг за шагом:

1. **Шаг 1:** Генерация схемы базы данных (Drizzle ORM) с правильными связями (Тенанты, Пиццерии, Шаблоны, Логи).
2. **Шаг 2:** Настройка Next.js App Router, авторизации (RBAC) и Layout-ов дашборда с фильтрацией по ролям.
3. **Шаг 3:** Разработка публичного PWA-интерфейса (сканер QR, выбор смены, мгновенное проставление галочек, сбор GPS).
4. **Шаг 4:** Разработка UI конструктора чек-листов (JSONB) и Дашборда аналитики с детекцией аномалий.

Выдавать план для AI?

вот теперь собери это все в единый документ-концепцию

КОНЦЕПЦИЯ СИСТЕМЫ ЦИФРОВЫХ ЧЕК-ЛИСТОВ (DODO CHECKLISTS)

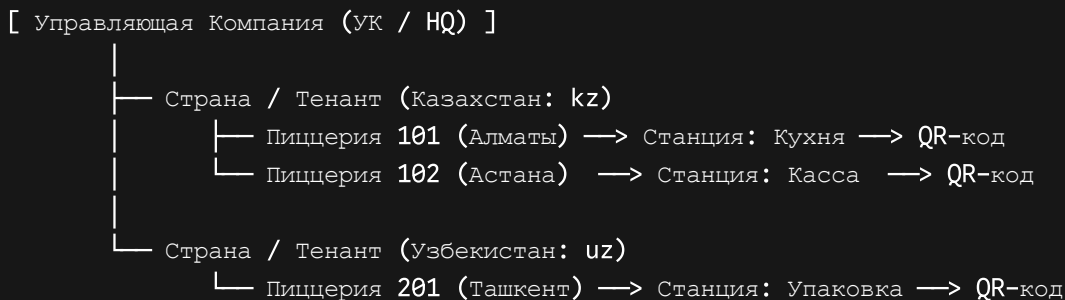
1. Обзор и бизнес-цель

Назначение системы: Цифровизация и ускорение операционного контроля ежедневных процессов в сети пиццерий.

Ключевая философия: Zero-Friction UX. Максимальная скорость работы линейного персонала на кухне без бюрократии, сборов паролей и блокировок, в сочетании с прозрачной аналитикой и детекцией аномалий для управляющих.

2. Доменная модель и Иерархия (Multi-Tenancy)

Иерархия обеспечивает жесткую изоляцию данных между странами и легкое управление масштабом сети.



- **Страна (Тенант):** Полностью изолированный контур данных со своим системным языком по умолчанию, справочником точек и чек-листами.
- **Форматы точек:** Support для стандартов **Full** (большие смены, разделение Утро/Вечер) и **Compact** (экспресс-точки на 2 человека).
- **Версионирование шаблонов:** Заполненные чек-листы сохраняют структуру шаблона в момент заполнения (JSON-снимок). Редактирование шаблона создаёт новую версию ($v1 \rightarrow v2$) и не искажает исторические отчеты.

3. Сценарии работы линейного персонала (Zero-Friction Flow)

[Сканирование QR] $\rightarrow$ [Авто-определение точки/станции] $\rightarrow$ [Выполнение] $\rightarrow$ [Фиксация]

1. **Доступ:** Повар или кассир сканирует QR-код станции с помощью камеры смартфона (или открывает его на планшете станции).
2. **Идентификация:** Авторизация не требуется. Система извлекает **Store ID** и **Station ID** из URL.

3. **Заполнение:** Открывается текущий чек-лист с авто-определением языка устройства. Сотрудник проставляет галочки, вводит числа или оставляет короткие комментарии.
4. **Антифрод (Мягкие флаги без блокировки):**

1. **Геолокация:** Браузер запрашивает GPS. Если координаты устройства выходят за геозону пиццерии (> 150 м), смене присваивается бейдж `Flagged: Geo Mismatch`.

2. **Скорость кликов:** Если чек-лист из 15 пунктов проставляется менее чем за 10 секунд, присваивается бейдж `Flagged: Batch Checked`.

3. **Задержка:** Открытие или закрытие утреннего чек-листа во второй половине дня маркируется как `Flagged: Late`.

4. Роли и Матрица прав доступа (RBAC)

Роль в системе	Права на администрирование	Права на редактор шаблонов	Права на анализ
Суперадмин УК (HQ)	Все страны и пользователи. Создание новых арендаторов.	Создание глобальных стандартов сети.	Вся сеть (все пиццерии)
Админ Страны / Партнера	Управление пользователями своей страны.	Адаптация и создание шаблонов для своей страны.	Все пиццерии страны.
Управляющий Точки	Скрыто	Скрыто	Только своя пиццерия.

...

5. Структура Веб-Интерфейса Панели Управления

Интерфейс динамически адаптируется под роль авторизованного пользователя.

Вкладка 1: Статистика (Дашборд)

- **Фильтры:** Выбор точки, станции, диапазона дат и статуса смены.

• **KPIs:** Процент вовремя закрытых смен, количество незакрытых критических инцидентов.

• **Лента смен (Shift Feed):** Карточки смен с цветовой индикацией:

Зеленый

 — без нарушений.

- Желтый/Оранжевый теги — индикаторы аномалий (Вне геозоны , Быстрое заполнение , Задержка).
- Детализация: Детальный просмотр заполненного чек-листа с фиксацией времени каждого клика и комментариев.

Вкладка 2: Менеджер чек-листов (Редактор)

- Простой блочный редактор шаблонов (без сложного Drag-and-Drop).
- Поля пункта:
 - Текст вопроса на нескольких языках (мультиязычные табы: RU , KK , UZ , EN).
 - Тип ответа: Да/Нет , Числовое значение (Мин/Макс) , Текстовый комментарий .
 - Переключатель Критичный пункт (требуется обязательно указать причину отмены и генерирует алерт).
- Кнопки «Сохранить черновик» и «Опубликовать новую версию».

Вкладка 3: Настройки

- Справочник пиццерий: Добавление точек, адресов, *Lat/Lon* GPS-координат и радиуса допустимого охвата.
- Генератор QR-кодов по станциям (Кухня, Касса, Упаковка).
- Настройка Webhook-коннекторов для внешней шины уведомлений.

Вкладка 4: Управление доступом (Администрирование)

(Видна только Суперадмину УК и Админу Страны)

- Создание и редактирование учетных записей менеджеров.
- Создание стран-тенантов и назначение языков по умолчанию (только для Суперадмина УК).

6. Техническая Архитектура и Стек

- Framework: Next.js (TypeScript, App Router, Server Actions) — единый TypeScript-стек.
- UI Kit: Tailwind CSS + shadcn/ui.
- БД и ORM: PostgreSQL + Drizzle ORM (native поддержка JSONB и типа Ltree для иерархий).
- Мультиязычность:
 - Системный UI (Кнопки, меню): next-intl (локальные JSON-файлы).
 - Контент чек-листов: Поля типа JSONB в PostgreSQL: `title: { "ru": "...", "kk": "..." }`.
- Шина уведомлений (Event Bus):
 - Паттерн Observer. При возникновении события (IncidentCreated , ShiftCompletedWithErrors) система вызывает зарегистрированные

7. Схема Базы Данных (Drizzle ORM)

TypeScript



```
import { pgTable, uuid, text, timestamp, jsonb, boolean, integer, doublePrecision } from 'drizzle-orm/pg-core';

export const countries = pgTable('countries', {
  id: text('id').primaryKey(), // 'kz', 'uz', 'ru'
  name: text('name').notNull(),
  defaultLanguage: text('default_language').notNull().default('ru'),
});

export const stores = pgTable('stores', {
  id: uuid('id').primaryKey().defaultRandom(),
  countryId: text('country_id').references(() => countries.id),
  name: text('name').notNull(),
  latitude: doublePrecision('latitude'),
  longitude: doublePrecision('longitude'),
  radiusMeters: integer('radius_meters').default(150),
});

export const checklistTemplates = pgTable('checklist_templates', {
  id: uuid('id').primaryKey().defaultRandom(),
  countryId: text('country_id').references(() => countries.id),
  version: integer('version').notNull().default(1),
  title: text('title').notNull(),
  station: text('station').notNull(),
  storeFormat: text('store_format').notNull(), // 'full' | 'compact'
  items: jsonb('items').notNull(), // [{ id, title: {ru, kk}, type: 'boolean'|'number', is...
  createdAt: timestamp('created_at').defaultNow(),
});

export const submissions = pgTable('submissions', {
  id: uuid('id').primaryKey().defaultRandom(),
  storeId: uuid('store_id').references(() => stores.id),
  templateId: uuid('template_id').references(() => checklistTemplates.id),
  station: text('station').notNull(),
  responses: jsonb('responses').notNull(), // [{ itemId, value, comment, clientTimestamp }
  flags: jsonb('flags').default([]), // ['GEO_MISMATCH', 'BATCH_CHECKED', 'LATE']
  userLat: doublePrecision('user_lat'),
  userLon: doublePrecision('user_lon'),
  startedAt: timestamp('started_at').notNull(),
  completedAt: timestamp('completed_at').defaultNow(),
});
```

8. Дорожная карта и Промпты для AI-Агента (Cursor / Devin)

Шаг 1: Инициализация и База Данных

Prompt for AI:

"Initialize a Next.js App Router project with TypeScript, Tailwind CSS, shadcn/ui, Drizzle ORM, and PostgreSQL setup. Create the database schemas for `countries`, `stores`, `checklist_templates`, and `submissions` as defined in the Drizzle schema specification. Include database migration scripts and a basic seed file for initial data."

Шаг 2: Модуль Авторизации и RBAC Layouts

Prompt for AI:

"Implement Auth.js (NextAuth) authentication with Role-Based Access Control (SUPERADMIN_HQ, PARTNER_ADMIN, STORE_MANAGER). Create sidebar layout navigation that dynamically shows/hides tabs based on the user's role: Statistics (all roles), Checklist Manager (HQ & Partner only), Settings (Store Manager sees QR printer only; HQ/Partner see store management), Admin (HQ & Partner only)."

Шаг 3: Публичный PWA-интерфейс сканирования и заполнения

Prompt for AI:

"Create a public client route `/scan` that takes `storeId` and `station` URL query parameters.

1. Fetch browser geolocation (`navigator.geolocation`) and detect language (`navigator.language`).
2. Fetch active `checklist_template` for the station. Render multi-language question titles based on device language falling back to tenant default language.
3. Allow checking items with dynamic responses (boolean, number, text). If item is marked `isCritical` and set to false/failed, require a short comment.
4. On submit, calculate timestamp deltas and compare GPS distance to store coordinates. Attach anomaly flags (`GEO_MISMATCH` , `BATCH_CHECKED`) and save submission to DB."

Шаг 4: Дашборд аналитики и Редактор шаблонов

Prompt for AI:

- "1. Build the Checklist Editor UI in the `/admin/templates` route allowing managers to create/edit template items with multi-language JSONB titles (RU, KK, UZ, EN), toggle response types, and set critical flags.
2. Build the Analytics Dashboard UI in `/dashboard` displaying a list of submissions with status badges for `GEO_MISMATCH` , `BATCH_CHECKED` , and `LATE` . Add filters for Country, Store, Station, and Date Range."

Markdown



КОНЦЕПЦИЯ СИСТЕМЫ ЦИФРОВЫХ ЧЕК-ЛИСТОВ (DODO CHECKLISTS)

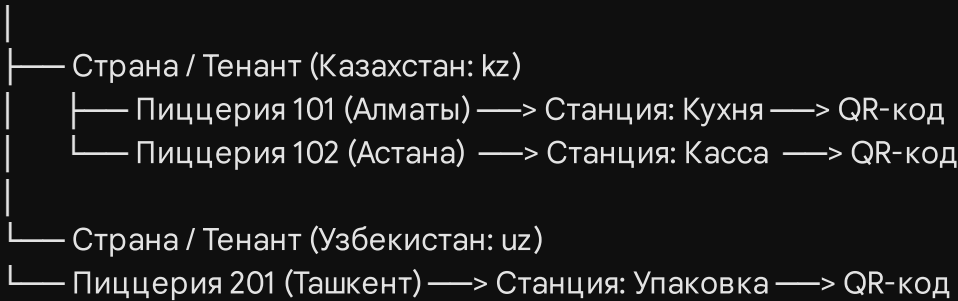
1. Обзор и бизнес-цель

****Назначение системы:**** Цифровизация и ускорение операционного контроля ежедневных процесс
****Ключевая философия:**** Zero-Friction UX. Максимальная скорость работы линейного персонала

2. Доменная модель и Иерархия (Multi-Tenancy)

Иерархия обеспечивает жесткую изоляцию данных между странами и легкое управление масштабом

[Управляющая Компания (УК / HQ)]



* ****Страна (Тенант):**** Полностью изолированный контур данных со своим системным языком по
* ****Форматы точек:**** Поддержка стандартов `Full` (большие смены, разделение Утро/Вечер) и
* ****Версионирование шаблонов:**** Заполненные чек-листы сохраняют структуру шаблона в момент

3. Сценарии работы линейного персонала (Zero-Friction Flow)

[Сканирование QR] —> [Авто-определение точки/станции] —> [Выполнение] —> [Фиксация]

1. ****Доступ:**** Повар или кассир сканирует QR-код станции с помощью камеры смартфона (или о
2. ****Идентификация:**** Авторизация не требуется. Система извлекает `Store ID` и `Station ID`
3. ****Заполнение:**** Открывается текущий чек-лист с авто-определением языка устройства. Сotr
4. ****Антифрод (Мягкие флаги без блокировки):****
 - * ****Геолокация:**** Браузер запрашивает GPS. Если координаты устройства выходят за геозон
 - * ****Скорость кликов:**** Если чек-лист из 15 пунктов проставляется менее чем за 10 секунд
 - * ****Задержка:**** Открытие или закрытие утреннего чек-листа во второй половине дня маркир

4. Роли и Матрица прав доступа (RBAC)

Роль в системе	Права на администрирование	Права на редактор шаблонов	Права на анал
:---	:---	:---	:---
Суперадмин УК (HQ)	Все страны и пользователи. Создание новых тенантов.	Создание	
Админ Страны / Партнера	Управление пользователями своей страны.	Адаптация и созд	
Управляющий Точки	*Скрыто*	*Скрыто*	**Только своя пиццерия.** **Только печат

5. Структура Веб-Интерфейса Панели Управления

Интерфейс динамически адаптируется под роль авторизованного пользователя.

Вкладка 1: Статистика (Дашборд)

- * ****фильтры:**** Выбор точки, станции, диапазона дат и статуса смены.
- * ****KPIs:**** Процент вовремя закрытых смен, количество незакрытых критических инцидентов.
- * ****Лента смен (Shift Feed):**** Карточки смен с цветовой индикацией:
 - * `Зеленый` – без нарушений.
 - * `Желтый/Оранжевый теги` – индикаторы аномалий (`Вне геозоны`, `Быстрое заполнение`, `З
- * ****Детализация:**** Детальный просмотр заполненного чек-листа с фиксацией времени каждого к

Вкладка 2: Менеджер чек-листов (Редактор)

- * Простой блочный редактор шаблонов (без сложного Drag-and-Drop).
- * ****Поля пункта:****
 - * Текст вопроса на нескольких языках (мультиязычные табы: `RU`, `KK`, `UZ`, `EN`).
 - * Тип ответа: `Да/Нет`, `Числовое значение (Мин/Макс)`, `Текстовый комментарий`.
 - * Переключатель `Критичный пункт` (требуется обязательно указать причину отмены и генериру
- * Кнопки «Сохранить черновик» и «Опубликовать новую версию».

Вкладка 3: Настройки

- * Справочник пиццерий: Добавление точек, адресов, GPS-координат (Lat/Lon) и радиуса допуст
- * Генератор QR-кодов по станциям (Кухня, Касса, Упаковка).
- * Настройка Webhook-коннекторов для внешней шины уведомлений.

Вкладка 4: Управление доступом (Администрирование)

- * (Видна только Суперадмину УК и Админу Страны)*
- * Создание и редактирование учетных записей менеджеров.
- * Создание стран-тенантов и назначение языков по умолчанию (только для Суперадмина УК).

6. Техническая Архитектура и Стек

- * **Framework:** Next.js (TypeScript, App Router, Server Actions) – единый TypeScript-стек
- * **UI Kit:** Tailwind CSS + shadcn/ui.
- * **БД и ORM:** PostgreSQL + Drizzle ORM (native поддержка JSONB и типа Ltree для иерархий)
- * **Мультиязычность:**
 - * **Системный UI** (Кнопки, меню): * `next-intl` (локальные JSON-файлы).
 - * **Контент чек-листов:** Поля типа `JSONB` в PostgreSQL: `title: { "ru": "...", "kk": "..."`
- * **Шина уведомлений (Event Bus):**
 - * Паттерн **Observer**. При возникновении события (`IncidentCreated`, `ShiftCompletedWithErr

7. Схема Базы Данных (Drizzle ORM)

```
````typescript
import { pgTable, uuid, text, timestamp, jsonb, boolean, integer, doublePrecision } from '

export const countries = pgTable('countries', {
 id: text('id').primaryKey(), // 'kz', 'uz', 'ru'
 name: text('name').notNull(),
 defaultLanguage: text('default_language').notNull().default('ru'),
});

export const stores = pgTable('stores', {
 id: uuid('id').primaryKey().defaultRandom(),
 countryId: text('country_id').references(() => countries.id),
 name: text('name').notNull(),
 latitude: doublePrecision('latitude'),
 longitude: doublePrecision('longitude'),
 radiusMeters: integer('radius_meters').default(150),
});

export const checklistTemplates = pgTable('checklist_templates', {
 id: uuid('id').primaryKey().defaultRandom(),
 countryId: text('country_id').references(() => countries.id),
 version: integer('version').notNull().default(1),
 title: text('title').notNull(),
 station: text('station').notNull(),
 storeFormat: text('store_format').notNull(), // 'full' | 'compact'
 items: jsonb('items').notNull(), // [{ id, title: {ru, kk}, type: 'boolean'|'number', is
 createdAt: timestamp('created_at').defaultNow(),
});

export const submissions = pgTable('submissions', {
 id: uuid('id').primaryKey().defaultRandom(),
 storeId: uuid('store_id').references(() => stores.id),
 templateId: uuid('template_id').references(() => checklistTemplates.id),
 station: text('station').notNull(),
 responses: jsonb('responses').notNull(), // [{ itemId, value, comment, clientTimestamp }
 flags: jsonb('flags').default([]), // ['GEO_MISMATCH', 'BATCH_CHECKED', 'LATE']
 userLat: doublePrecision('user_lat'),
 userLon: doublePrecision('user_lon'),
 startedAt: timestamp('started_at').notNull(),
```

```
completedAt: timestamp('completed_at').defaultNow(),
 });
```

## 8. Дорожная карта и Промпты для AI-Агента (Cursor / Devin)

### Шаг 1: Инициализация и База Данных

#### Prompt for AI:

"Initialize a Next.js App Router project with TypeScript, Tailwind CSS, shadcn/ui, Drizzle ORM, and PostgreSQL setup. Create the database schemas for `countries`, `stores`, `checklist_templates`, and `submissions` as defined in the Drizzle schema specification. Include database migration scripts and a basic seed file for initial data."

### Шаг 2: Модуль Авторизации и RBAC Layouts

#### Prompt for AI:

"Implement Auth.js (NextAuth) authentication with Role-Based Access Control (SUPERADMIN\_HQ, PARTNER\_ADMIN, STORE\_MANAGER). Create sidebar layout navigation that dynamically shows/hides tabs based on the user's role: Statistics (all roles), Checklist Manager (HQ & Partner only), Settings (Store Manager sees QR printer only; HQ/Partner see store management), Admin (HQ & Partner only)."

### Шаг 3: Публичный PWA-интерфейс сканирования и заполнения

#### Prompt for AI:

"Create a public client route `/scan` that takes `storeId` and `station` URL query parameters.

1. Fetch browser geolocation ( `navigator.geolocation` ) and detect language ( `navigator.language` ).
2. Fetch active `checklist_template` for the station. Render multi-language question titles based on device language falling back to tenant default language.
3. Allow checking items with dynamic responses (boolean, number, text). If item is marked `isCritical` and set to false/failed, require a short comment.
4. On submit, calculate timestamp deltas and compare GPS distance to store coordinates. Attach anomaly flags ( `GEO_MISMATCH` , `BATCH_CHECKED` ) and save submission to DB."

### Шаг 4: Дашборд аналитики и Редактор шаблонов

#### Prompt for AI:

"1. Build the Checklist Editor UI in the `/admin/templates` route allowing managers to create/edit template items with multi-language JSONB titles (RU, KK, UZ, EN), toggle response types, and set critical flags.

2. Build the Analytics Dashboard UI in `/dashboard` displaying a list of submissions with status badges for `GEO_MISMATCH`, `BATCH_CHECKED`, and `LATE`. Add filters for Country, Store, Station, and Date Range."